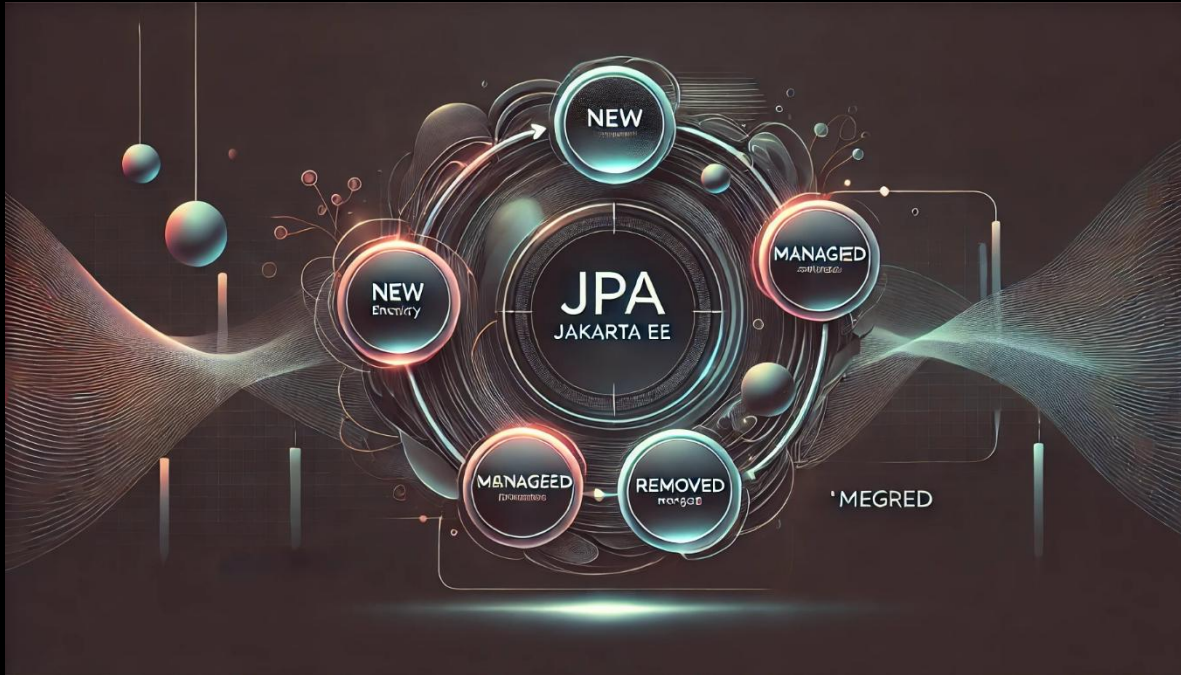


Eliminar un Objeto con JPA y Jakarta EE



Eliminar un Objeto con JPA en Jakarta EE

En este ejercicio veremos cómo eliminar un objeto de la base de datos utilizando **JPA**. La eliminación se realiza en dos transacciones:

1. **Recuperar el objeto a eliminar** mediante una consulta `SELECT`.
2. **Eliminar el objeto** en una segunda transacción.

1 Implementación de la Eliminación con JPA

A continuación, presentamos el código de la clase `EliminarObjetoJPA`, con las mejores prácticas aplicadas.

Código de `EliminarObjetoJPA.java`

```
package sga.cliente.ciclodevida;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.EntityTransaction;
import jakarta.persistence.Persistence;
```

```
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class EliminarObjetoJPA {

    private static final Logger log = LoggerFactory.getLogger(EliminarObjetoJPA.class);

    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");

        try (EntityManager em = emf.createEntityManager()) {
            // Paso 1: Iniciar la primera transacción
            EntityTransaction tx = em.getTransaction();
            tx.begin();

            // Paso 2: Buscar el objeto a eliminar
            Persona persona1 = em.find(Persona.class, 4);

            // Paso 3: Finalizar la primera transacción
            tx.commit();

            // Validar si el objeto existe antes de intentar eliminarlo
            if (persona1 == null) {
                log.warn("El objeto con ID 4 no existe en la base de datos.");
                return;
            }

            log.info("Objeto encontrado: " + persona1);

            // Paso 4: Iniciar la segunda transacción
            EntityTransaction tx2 = em.getTransaction();
            tx2.begin();

            // Paso 5: Eliminar el objeto
            em.remove(em.contains(persona1) ? persona1 : em.merge(persona1));

            // Paso 6: Finalizar la segunda transacción
            tx2.commit();

            log.info("Objeto eliminado en estado de detached: " + persona1);
        }
    }
}
```

2 Explicación del Código

1. **Paso 1:** Se inicia una transacción (`tx.begin()`).
2. **Paso 2:** Se recupera el objeto con `find()` usando el ID 4.
3. **Paso 3:** Se finaliza la transacción (`tx.commit()`), dejando el objeto en estado **detached**.
4. **Validación:** Se verifica si el objeto existe antes de intentar eliminarlo.
5. **Paso 4:** Se inicia una nueva transacción para la eliminación.
6. **Paso 5:** Se elimina el objeto con `remove()`, pero antes se usa `merge()` en caso de que el objeto esté **detached**.
7. **Paso 6:** Se finaliza la transacción con `tx2.commit()`.

Vamos a desglosar la línea:

```
em.remove(em.contains(persona1) ? persona1 : em.merge(persona1));
```

Esta línea es responsable de **eliminar un objeto de la base de datos en JPA**. Vamos a explicarlo paso por paso:

◆ Explicación Paso a Paso

1. `em.contains(persona1)`

- `contains()` verifica si el objeto `persona1` **ya está administrado por el EntityManager**.
- Si el objeto **está en el estado "managed"**, significa que está sincronizado con la base de datos.
- **Devuelve true si el objeto es administrado**, y `false` si es "detached".

2. `persona1 : em.merge(persona1)` (Operador ternario `? :`)

- **Si `persona1` está en estado "managed"** (`contains(persona1) == true`), se usa `persona1` directamente.
- ****Si `persona1` está en estado "detached"****, significa que ya no está gestionado por JPA, así que necesitamos volver a ****adjuntarlo**** al contexto de persistencia con `merge(persona1)`.
- `merge(persona1)` devuelve un **nuevo objeto gestionado**, que será el que eliminemos.

3 `em.remove(...)`

- `remove()` elimina el objeto de la base de datos, **pero solo puede eliminar objetos en estado "managed"**.
- Si el objeto ya estaba administrado (`contains() == true`), lo elimina directamente.
- Si no estaba administrado (`contains() == false`), primero lo convierte a "managed" con `merge()` y luego lo elimina.

3 Ejecución y Verificación

Para comprobar que la eliminación se realizó correctamente:

1. **Ejecutar la clase `EliminarObjetoJPA`.**
2. **Observar la salida en los logs** para asegurarse de que el objeto fue encontrado y eliminado correctamente.

```
[EL Fine]: sql: ServerSession(1370074462)--Connection(553672556)--SELECT
id_persona, APELLIDO, EMAIL, NOMBRE, TELEFONO FROM PERSONA WHERE (id_persona = ?)
```

```
bind => [4]
```

```
[main] INFO sga.cliente.ciclodevida.EliminarObjetoJPA - Objeto encontrado:
Persona{idPersona=4, nombre=Pedro, apellido=Luna, email=pluna@mail.com,
telefono=55998877}
```

```
[EL Fine]: sql: ClientSession(907815246)--Connection(553672556)--DELETE FROM
PERSONA WHERE (id_persona = ?)
```

```
bind => [4]
```

```
[main] INFO sga.cliente.ciclodevida.EliminarObjetoJPA - Objeto eliminado en
estado de detached: Persona{idPersona=4, nombre=Pedro, apellido=Luna,
email=pluna@mail.com, telefono=55998877}
```

3. **Ejecutar esta consulta SQL en MySQL** para verificar que el objeto ya no existe:

```
SELECT * FROM persona WHERE id_persona = 4;
```

Si todo funciona correctamente, el **registro con ID 4 debería haber sido eliminado de la base de datos**.

¡Y eso es todo! Ahora sabes cómo eliminar un objeto en JPA con transacciones separadas y buenas prácticas. 🚀

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)